

DevOps

Development and Operations

Overview

- Introduction to DevOps
- Continuous Integration / Continuous Delivery (CI/CD)

Learning Objectives

- Explain the role DevOps plays in modern software
- Identify the role CI/CD plays in modern software
- Create a Continuous Integration workflow with GitHub Actions

DevOps - A History

Developers and IT Ops professionals had separate (and often competing) objectives, department leadership, key performance indicators, and often worked on separate floors or even separate buildings.

Developers looked after building the system, IT Ops looked after supporting and monitoring the system.

This resulted in siloed teams concerned only with their processes and releases.

These silos often meant miscommunication, delayed deliveries and strains on morale and relationships.

What is DevOps?

A combination of **culture, practises** and **tools** that increases an organisation's ability to deliver services at high velocity.

More succinctly, it is a set of practices that combines software development (Dev) and IT operations (Ops).

Under a DevOps model, dev and ops are no longer siloed.

Both are merged into a single team where engineers work across the entire application lifecycle, from development and test to deployment to operations, and develop a range of skills not limited to a single function.

DevOps Culture

- **Increased collaboration** as a team as opposed to silos
- **Share responsibility** across the whole team so no one process is looked after by specific people, thus spreading the knowledge and pain
- **No silos** between development and operations
- Give power to **autonomous teams** to enable them to make their own decisions in order to collaborate effectively, and remove convoluted decision making processes
- **Build quality into the development process**
- **Value feedback** to continuously improve ways of working
- **Automate** as much as you can

DevOps Practices

- Infrastructure as Code
- Continuous Integration (CI)
- Continuous Delivery (CD)
- Smaller Apps and Web Servers (services) that are decoupled
- Monitoring and Logging
- Communication and Collaboration

In this module, we will be looking into Continuous Integration (CI), and Continuous Delivery (CD).

DevOps Tools

- Source Control (Git)
- Collaboration / communication tools (Slack, Jira, Trello)
- Issue tracking (Jira, Trello, ZenDesk, MS Planner, GitHub Projects)
- Configuration tools (Puppet, Chef)
- CI/CD tools (we will come onto these)
- And plenty more!

Benefits of DevOps

Speed: Smaller apps and continuous delivery lets teams take ownership of services and then release updates to them quicker.

Rapid delivery: Increase the frequency and pace of releases so you can innovate and improve your product faster.

The quicker you can release new features and fix bugs, the faster you can respond to your customers' needs and build competitive advantage.

Reliability: Quality of application updates and infrastructure changes so you can reliably deliver at a faster pace while maintaining a positive experience for end users.

Better internal culture: DevOps practises lead to better communication, increased productivity and agility.

Why does software need to change?

It changes all the time for many reasons;

- Features
- Bug fixes
- Security patches
- Contract changes
- Performance optimisations

What does software need to be able to operate?

Many parts might be required;

- Server
- Database
- Cache
- Storage
- APIs
- Networking

What is an environment?

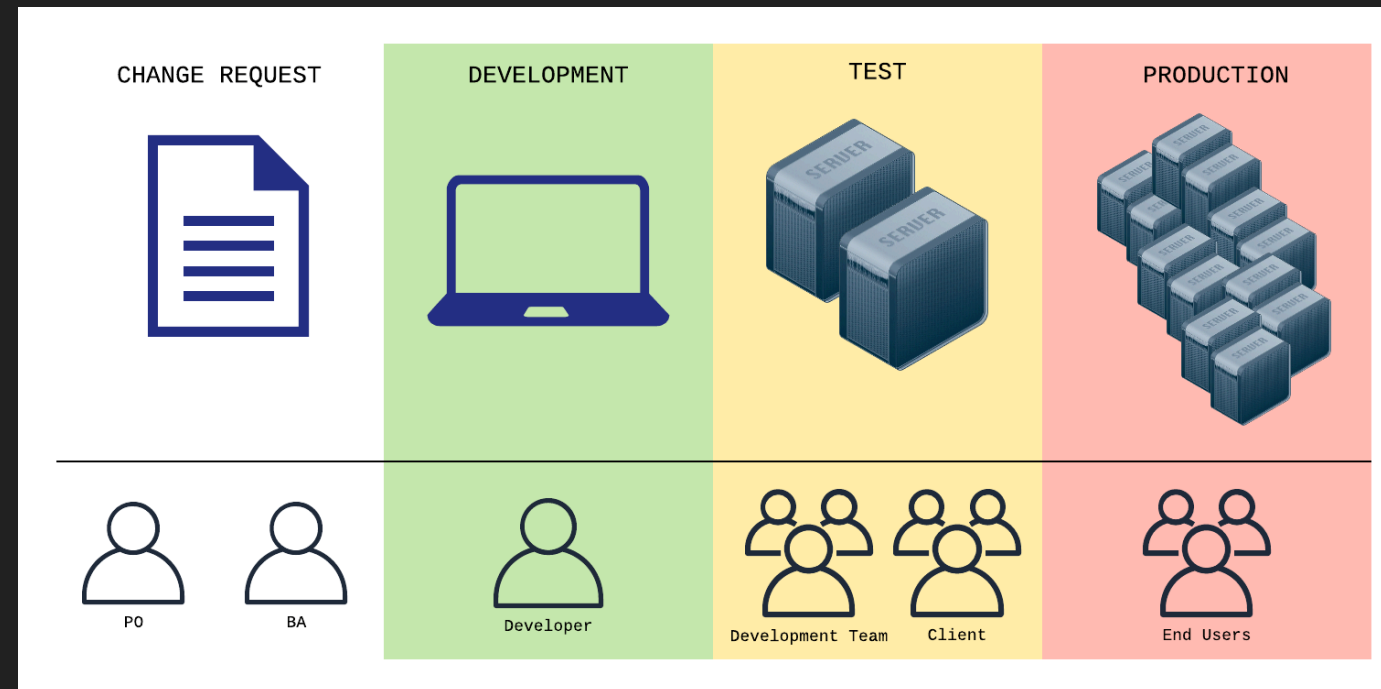
An **environment** is the place where our software runs.

Environments are comprised of the resources and infrastructure that support the operation of a software system.

Who needs access to an environment?

- Developers
- Operations
- Testers
- Product Owner
- Client's support team
- End users

Software Lifecycle



The development environment is usually the first thing you set up when working on a software project

Types of environments

Development: maximise developer productivity, usually local.

Testing: hosted, similar to customer-facing environment, for more complex integration and E2E tests.

UAT/Staging: as stable and production-like as possible, used for customer demos and wider service integration.

Production: live, end-user-facing environment. *You must protect it at all costs.*

Different people scrutinising the product at various stages reduces the risk of severe defects.

Promoting software through our environments

Having multiple environments helps find bugs in our software before it's released to the public.

Changes to the software require deployments up the chain, all the way to production.

A recipe to deploy software reliably

- Make change
- Build code
- Run unit tests
- Run code quality metrics
- Install the required dependencies in the target environment
- Install our software in the target environment

Maintaining environments is hard work

- Repetition without automation = more manual work
- Manual work = time wasted
- Manual work = potential differences in environments
- Differences in environments = constant breakage
- Constant breakage = downtime + longer wait to go live

How can we best alleviate this?

Quiz Time! 🧐

Which statement best describes DevOps?

1. Developers taking over all operations tasks.
2. Automating the process of software delivery and infrastructure changes.
3. The collaboration and communication of both software developers and other IT professionals while automating the process of software delivery and infrastructure changes.
4. The collaboration and communication of just software developers and operations staff while automating the process software delivery and infrastructure changes.

Answer: 3

Complete the following sentence with the best matching answer.

One goal of DevOps is to establish an environment where...

1. Change management does not control application releases.
2. Releasing more reliable applications faster and more frequently can occur.
3. Application development performs all operations tasks.
4. Releasing applications is valued over the quality of the released application.

Answer: 2

**Agile and DevOps are similar but differ in a few important aspects.
Which statement is correct?**

1. Agile is a change of thinking, DevOps is a cultural change in an organisation.
2. Agile is cultural change in an organisation, DevOps is a change of thinking.
3. Agile is process driven, DevOps is role driven.
4. Agile is role driven, DevOps is process driven.

Answer: **1**

Continuous Integration



What is CI?

Continuous Integration (CI) is a development practice that requires developers to integrate code into a shared repository several times a day. Each check-in is then verified by an automated build, allowing teams to detect problems early. By integrating regularly, you can detect errors quickly, and locate them more easily.

-- *ThoughtWorks*

CI has three stages of workflow.

1. Integrating changes to the main branch (every day)

- All about ensuring code gets into the main branch, as that branch is used for releasing software
- Easiest to do technically, hardest to do culturally
- Also known as trunk-based development
- Integrating code continually means branches are short-lived, which means code is going into main faster

2. Rely on automated tests

- Run an automated suite of tests on each integration to the main branch
- Instantly gives you information about failing tests, therefore failing software
- Can help spot bugs or errors
- Only run tests that are important, keep build time low

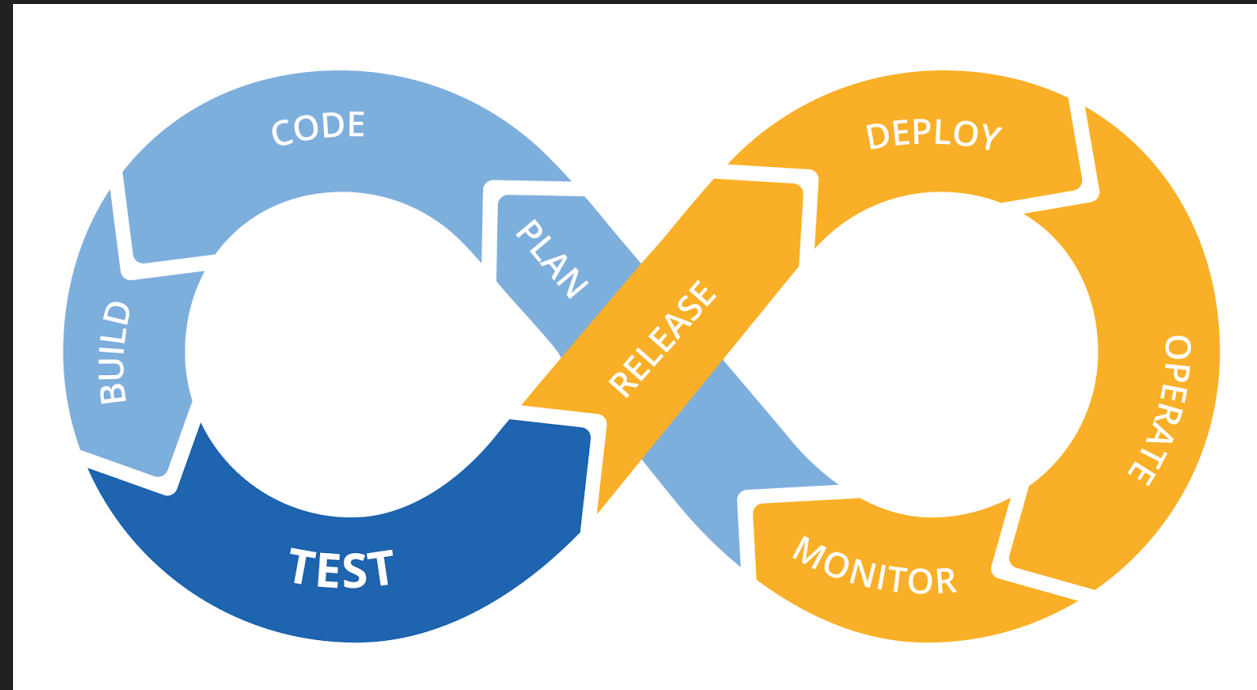
3. Prioritise broken builds

- If a build fails (code didn't compile, tests failed etc.), make fixing it the priority before doing any other tasks
- Everyone can see the same broken build which means better visibility of when something goes wrong

The enemies of CI

- Knowledge silos
- Manual work
- Inconsistent environments
- Big PRs

Continuous Deployment



What is CD?

- A way of getting your changes (features, bug fixes etc.) into production in a quick and sustainable way
- Code is checked to be in a deployable state by the CI stage

Principles of CD

Frequent, small deployments: Deploying smaller changes rather than an accumulation of changes means less chance of something going wrong.

Automation: Computers are infinitely better at repetitive tasks. Deployment steps can be automated so that we reduce human errors. Things like Infrastructure as Code (IaC) can help.

Keep improving: Always look to improve the deployment process. Are you manually doing anything that can be automated?

Shared responsibility: Everyone in the team is responsible for creating safe, fast and deterministic delivery of the deployed product, it shouldn't be siloed between devs and ops.

CI/CD Pipelines

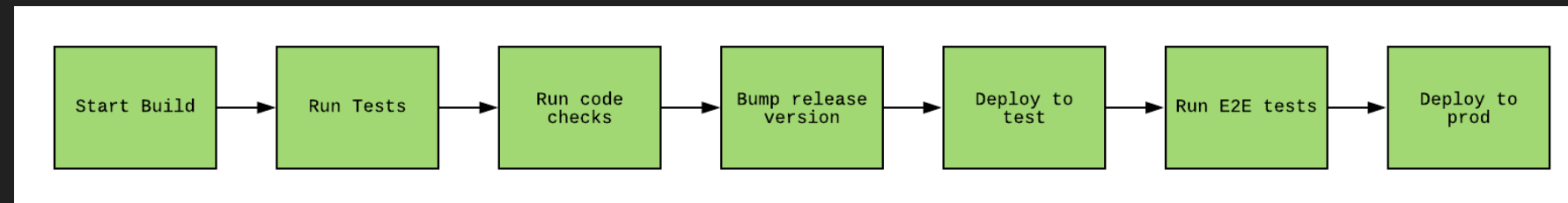


What is a pipeline?

A route, channel, or process along which something passes or is provided at a steady rate.

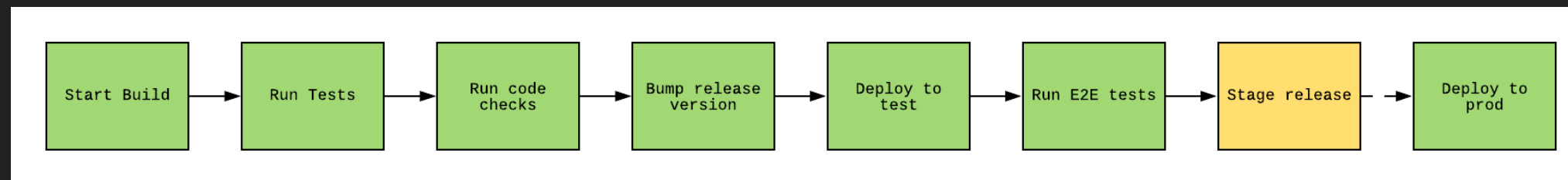
Build Pipeline

- Provides a steady supply of changes to our end-users
- Automates the integration and delivery flow of your software



Gated Releases

- A stage in the build pipeline that must be triggered manually to deploy to the next stage
- Allows you to ensure everything is how it should be in the stage before deploying to production



CI/CD Terms

Pipeline: automated stream of work that verifies, integrates and deploys software.

Build: a full cycle through the pipeline, triggered when a change is merged into the repo's stable branch.

Job: an individual step in the pipeline that carries out one or more tasks.

Task: a script run in a job.

CI/CD Software

- GitHub Actions
- CircleCI
- Jenkins
- Concourse
- And plenty more!

YAML

YAML Ain't a Markup Language.

A markup language designed primarily for humans rather than computers.

Structurally similar to JSON, except indentation defines the structure, like in Python.

YAML example

```
# Strings don't need quotes (this is a comment BTW)
key1: hello
key2:
  # You can nest keys
  subkey1: 1
  subkey2: hello
  # You can also make lists
  subkey3:
    - listitem1
    - listitem2
    - listitem3
```


GitHub Actions Example

Let's have a look together at the file `devops/handouts/python-project-example.yml`

This file shows how we might setup a GitHub action to run things for a python project.

Demo: GitHub Actions

Creating a new GitHub Actions workflow from scratch

Exercise prep



Instructor to give out the zip file of the exercise for DevOps
Everyone please unzip the file

Exercise: Getting started with GitHub Actions

From the zip, you should have file `exercises/ci-cd-exercise.md`

Let's all do the "DevOps CI/CD Exercise"

Emoji Check:

Do you feel you understood the key points of CI/CD?

1. 😓 Haven't a clue, please help!
2. 😞 I'm starting to get it but need to go over some of it please
3. 😐 Ok. With a bit of help and practice, yes
4. 😊 Yes, with team collaboration could try it
5. 😄 Yes, enough to start working on it collaboratively

Quiz Time! 🧐

What is meant by Continuous Integration?

1. A way of getting code changes into production in a quick and sustainable way.
2. A way of releasing new changes to your customers quickly in a sustainable way.
3. A way of requiring developers to push code into a shared repository several times a day. Each check-in is then verified by an automated build.
4. An architectural style that integrates an application as a collection of small autonomous services.

Answer: 3

What is meant by Continuous Deployment?

1. A way of getting code changes into production in a quick and sustainable way.
2. A way of releasing new changes to your a user acceptance environment quickly in a sustainable way.
3. A way of releasing new changes into a development environment in a quick and sustainable way.
4. An architectural style that integrates an application as a collection of small autonomous services.

Answer: **1**

Terms and Definitions - recap

DevOps: A set of practices that combines software development (Dev) and IT operations (Ops).

Continuous Integration: The practice of merging all developers' working copies to a shared mainline several times a day.

Continuous Delivery: A software engineering approach in which teams produce software in short cycles, ensuring that the software can be reliably released at any time and, when releasing the software, doing so automatically.

Terms and Definitions - recap

Environment: A computer system in which a computer program or software component is deployed and executed.

Build Pipeline: The process of taking code from version control and making it readily available to users of your application in an automated fashion.

Overview - recap

- Introduction to DevOps
- Continuous Integration / Continuous Delivery (CI/CD)

Learning Objectives - recap

- Explain the role DevOps plays in modern software
- Identify the role CI/CD plays in modern software
- Create a Continuous Integration workflow with GitHub Actions

Further Reading

- [DevOps Culture](#)
- [State of DevOps Report 2020](#)
- [The Phoenix Project \(DevOps Novel\)](#)
- [Accelerate, The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations \(Book\)](#)

Emoji Check:

On a high level, do you think you understand the main concepts of this session? Say so if not!

1. 🥲 Haven't a clue, please help!
2. 😞 I'm starting to get it but need to go over some of it please
3. 😐 Ok. With a bit of help and practice, yes
4. 😊 Yes, with team collaboration could try it
5. 😄 Yes, enough to start working on it collaboratively